Оглавление

БЛАНК ИНДИВИДУАЛЬНОГО ЗАДАНИЯ 2

Цель работы 3

Задание на учебную ознакомительную практику 3

1 РЕШЕНИЕ НЕЛИНЕЙНЫХ И ТРАНСЦЕНДЕНТНЫХ УРАВНЕНИЙ 4

1.1 Математическая постановка задачи 4

1.2 Описание метода численной реализации задания 4

1.3 Описание ручного счета тестового примера 5

1.4 Результат работы программы 8

2 РЕШЕНИЕ СЛАУ 9

2.1 Математическая постановка задачи 9

2.2 Описание метода численной реализации задания 9

2.3 Описание ручного счета тестового примера 10

2.4 Результат работы программы 18

ТЕСТИРОВАНИЕ РАЗРАБОТАННОЙ ПРОГРАММЫ 20

ЗАКЛЮЧЕНИЕ 29

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 30

ПРИЛОЖЕНИЕ 1 31

БЛАНК ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

на прохождение учебной практики обучающегося направления подготовки/ специальности 09.03.04 Программная инженерия

1 Фамилия, имя, отчество обучающегося:

2 Группа: 4333К

3 Тема индивидуального задания:
Применение численных методов при решении прикладных задач

4 Исходные данные:
MATLAB

Нелинейное уравнение по варианту

СЛАУ по варианту

5 Содержание отчетной документации:
5.1 индивидуальное задание;
5.2 отчёт, включающий в себя:
- титульный лист;
- материалы о выполнении индивидуального задания (содержание определяется кафедрой);
- выводы по результатам практики;
- список использованных источников.

5.3 отзыв руководителя от профильной организации (при прохождении практики в профильной организации).

6 Срок представления отчета на кафедру: «31» мая 2024

Руководитель практики
канд.тех.наук, доцент _____ А.А.Фоменкова
должность, уч. степень, звание подпись, дата инициалы, фамилия

СОГЛАСОВАНО
Руководитель практики от профильной организации
_____ _____ _____
должность подпись, дата инициалы, фамилия

Задание принял к исполнению
обучающийся
5.02.2024 _____
дата подпись инициалы, фамилия

Санкт–Петербург 2024

Цель работы

Закрепление теоретических знаний, полученных на лекционных, лабораторных и практических занятиях, развитие навыков самостоятельной работы по анализу информационных материалов печатных и электронных источников, закрепление навыков программирования на языке MATLAB, освоение работы с функциями и массивами данных, получение навыков построения интерфейса пользователя.

Задание на учебную ознакомительную практику

Разработать приложение с текстовым или графическим интерфейсом пользователя, реализующее алгоритмы решения:

1. нелинейного уравнения;
2. системы линейных алгебраических уравнений (СЛАУ);

заданным по варианту численным методом. Реализация алгоритма выполняется в виде функций на языке программирования MATLAB в среде MATLAB, FreeMat, Octave или SciLab.

Результаты оформляются в виде программы на языке программирования MATLAB и отчета, представляемого в конце практики.

1 РЕШЕНИЕ НЕЛИНЕЙНЫХ И ТРАНСЦЕНДЕНТНЫХ УРАВНЕНИЙ

1.1 Математическая постановка задачи

Пользователь вводит функцию и отрезок, где существует корень уравнения, все данные проверяются на корректность ввода и проверяется наличие корня. При корректном вводе, высчитывается корень функции методом хорд. Программа также находит значение корня используя встроенную функцию fzero(). Кроме вывода значений, выводится время выполнения обоих функций. Строится график, отмечается корень и границы отрезка. В таблице 1 представлены данные для тестового решения задачи.

Таблица 1 – Тестовый пример

№ варианта	Уравнение	Метод решения	Метод отделения корней
4	$3x - \cos x - 2 = 0$	Метод хорд	графический

1.2 Описание метода численной реализации задания

Идея способа хорд состоит в том, что можно с известным приближением допустить, что функция на достаточно малом участке [a, b] изменяется линейно. Тогда кривую y = f(x) на участке [a, b] можно заменить хордой и в качестве приближенного значения корня принять точку пересечения хорды с осью абсцисс. На рисунке 1 представлено графическое представление решения уравнения методом хорд.

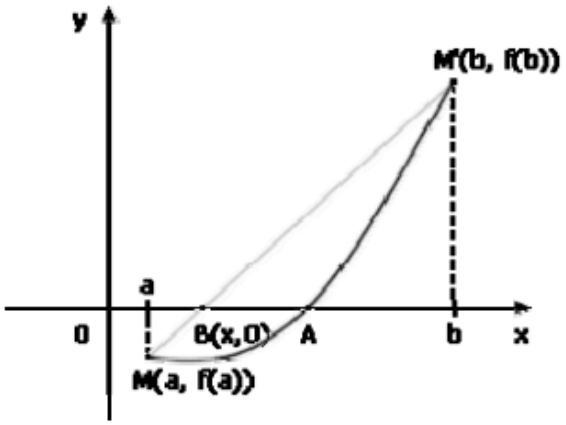


Рисунок 1 – Метод хорд графически

Построим график функции y = f(x) на участке [a, b]. Истинный корень уравнения f(x) = 0 есть абсцисса точки A, являющейся точкой пересечения кривой MM' с осью абсцисс.

Заменив кривую MM' хордой MM', мы примем в качестве приближенного значения корня абсциссу точки B, в которой хорда пересекается с осью.

Напишем уравнение прямой, проходящей через точки M(a, f(a)) и M'(b, f(b)):

$$\frac{y - f(a)}{f(b) - f(a)} = \frac{x - a}{b - a}$$

Абсцисса точки B, являющаяся приближенным корнем x1 уравнения f(x) = 0, может быть найдена из уравнения прямой, если положить в нем y = 0. Тогда получим

$$x = a - \frac{(b - a) * f(a)}{f(b) - f(a)}$$

Полученное значение x1 можно снова использовать для дальнейшего уточнения корня по способу хорд, рассматривая интервал [a, x1] или же [x1, b], смотря по тому, в каком из них лежит истинный корень. Чтобы определить это, находят знак f(x1).

1.3 Описание ручного счета тестового примера

В качестве начального интервала возьмём [0; pi], из графика заданной функции видно, что на нём существует корень. На рисунке 2 изображён график функции тестового выражения.

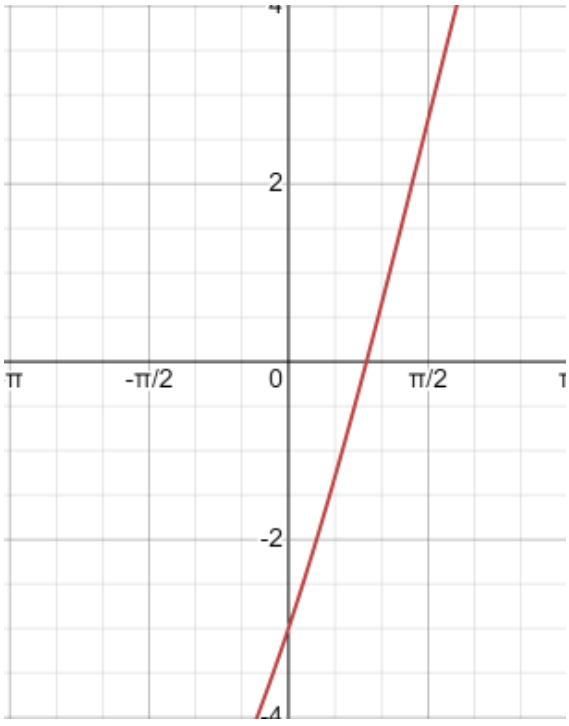


Рисунок 2 – График функции 3x-cosx-2

Выберем Epsilon = 0.001

1 Итерация.

$$x = 0 - \frac{(pi - 0) * (-3)}{8{,}42 - (-3)} = \frac{9.42}{11.42} = 0{,}8249$$

Смещение промежутка до [0,8248; pi]

2 Итерация.

$$x = 0{,}8249 - \frac{(pi - 0{,}8249) * (-0{,}20393)}{8.42 + 0{,}20393} = 0{,}8796$$

0,8796-0,8249>0.001

Смещение промежутка до [0,8796; pi]

3 Итерация.

$$x = 0{,}8796 - \frac{(pi - 0{,}8796) * (0{,}00134061)}{8.42 - 0{,}00134061} = 0{,}87924$$

0,8796-0,87924<0.001

x = 0,87924

1.4 Разработка алгоритмов решения задачи

Функция рассчитывает корень уравнения методом хорд.

Входные данные:

- Xmin, Xmax – Промежуток поиска
- func – Анонимная функция с самим выражением
- Epsilon – Допустимая погрешность

Выходные данные:

- x_new – Корень уравнения

  В случае отсутствия корней вернёт “NO_SOL”

На рисунке 3 представлена блок-схема алгоритма решения уравнения методом хорд.

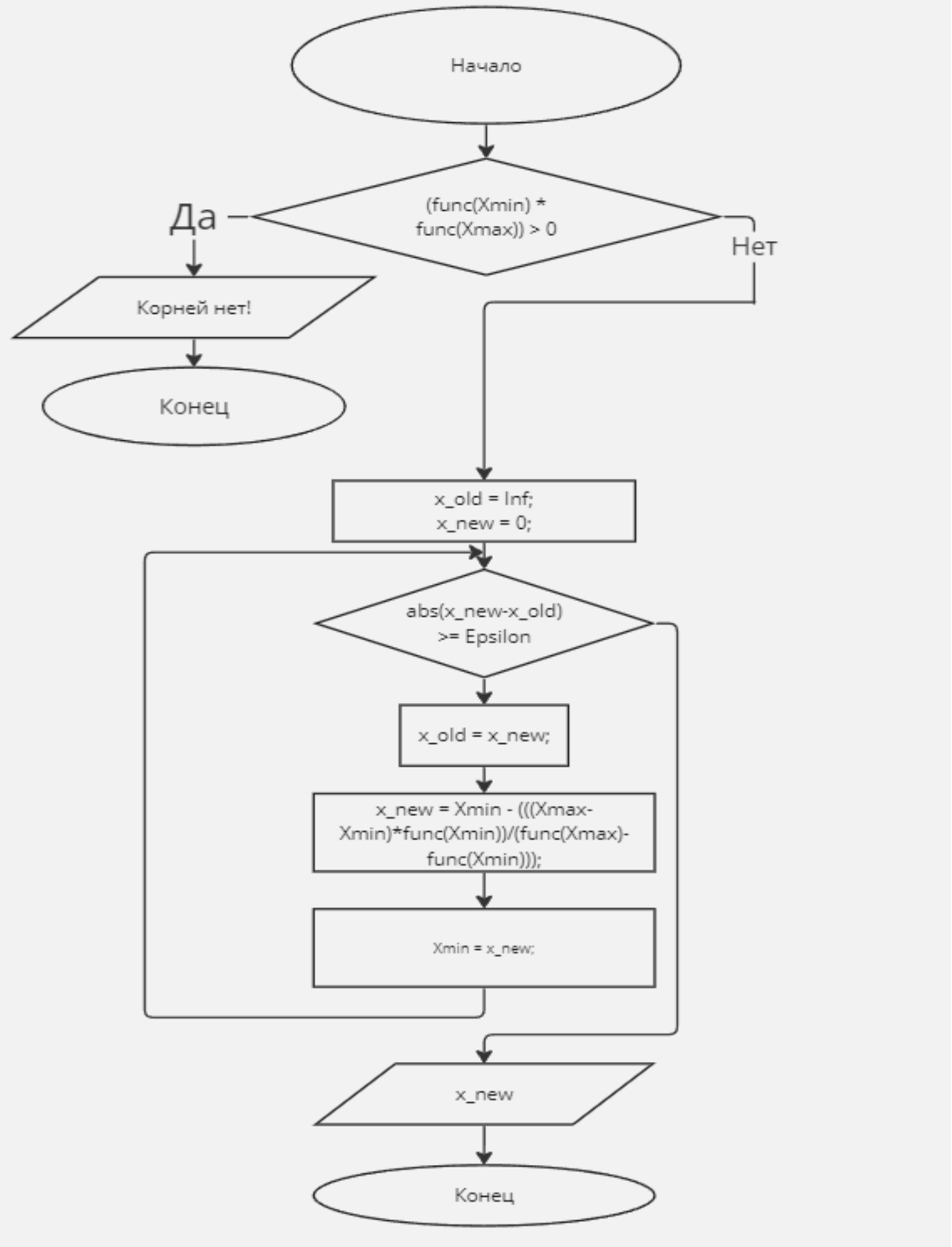


Рисунок 3 – Блок-схема решения уравнения методом хорд

1.4 Результат работы программы

На рисунке 4 изображена работа программы при выборе демонстрационного варианта.

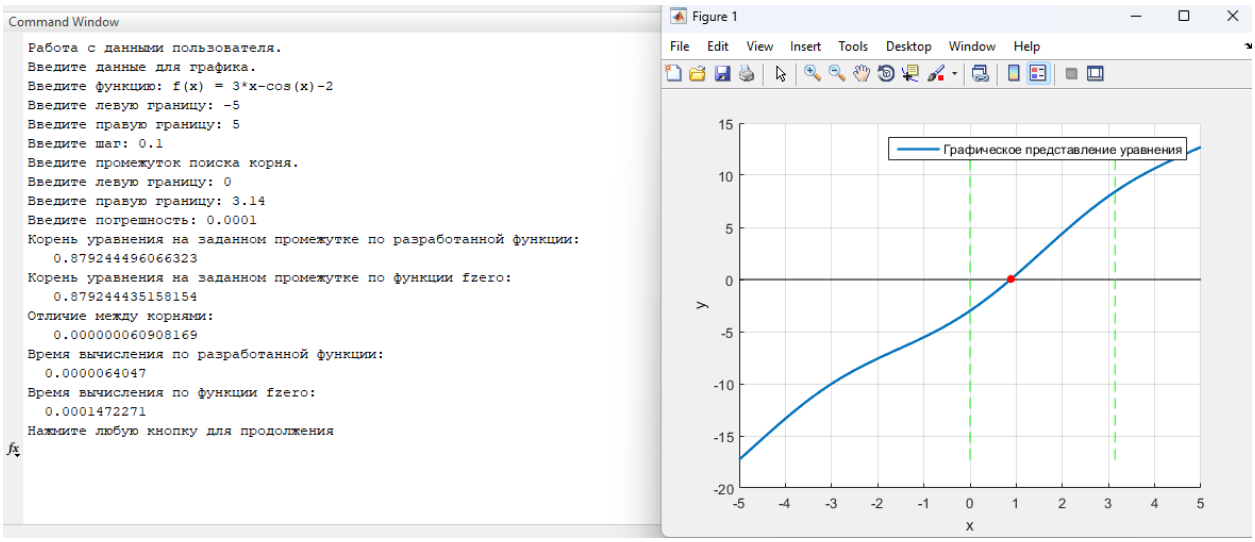


Рисунок 4 – Пример работы программы на тестовом уравнении

На рисунке 5 изображена работа программы при использовании данных пользователя.

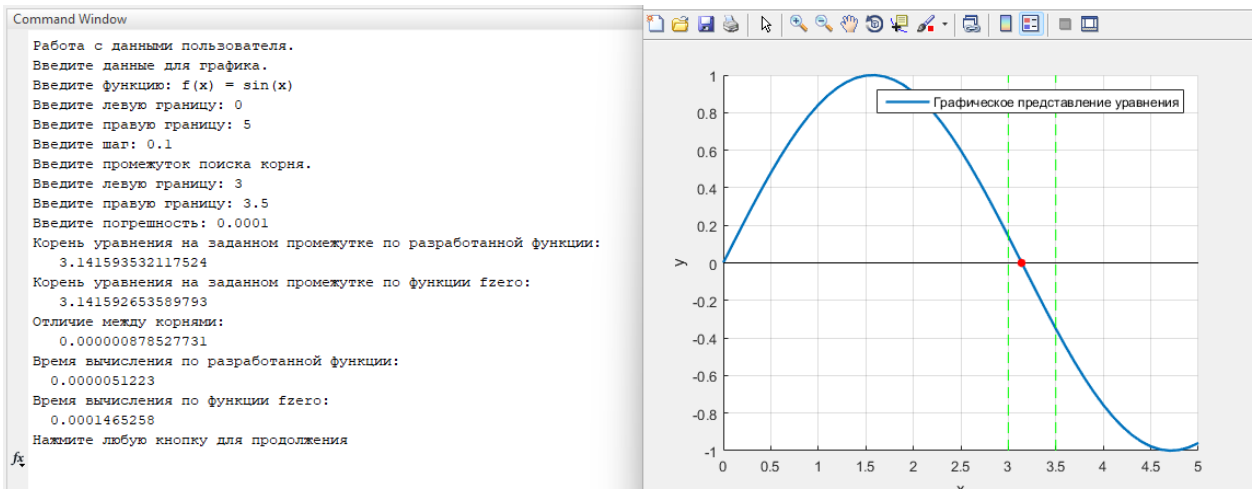


Рисунок 5 – Пример работы программы на данных пользователя

2 РЕШЕНИЕ СЛАУ

2.1 Математическая постановка задачи

Пользователь вводит количество уравнений, коэффициенты при неизвестных в СЛАУ и свободные члены. Все введенные числа проверяются на корректность. Идёт проверка на то, имеет ли матрица единственное решение. СЛАУ решается методом LU разложения и встроенной функцией lu() с последующим обратным делением. Кроме вывода корней СЛАУ, выводится время затраченное на LU разложение и выполнение lu(). В таблице 2 представлены данные для тестового решения задачи.

Таблица 2 – Тестовый пример

№ вар-та	СЛАУ	Метод решения
4	$\begin{cases} 2x_1+8x_2+7x_3+10x_4=17,8 \\ 5x_1+8x_2+2x_3+4x_4=10,5 \\ 5x_1+3x_2+2x_3+6x_4=10,1 \\ 7x_1+7x_2+5x_3+3x_4=12,7 \end{cases}$	Метод LU разложения

2.2 Описание метода численной реализации задания

Представим матрицы коэффициентов A как произведение двух матриц:

$$A = LU$$

L – Нижнеугольная матрица вида

$$L = \begin{bmatrix} l_{11} & 0 & 0 & \dots & 0 \\ l_{21} & l_{22} & 0 & \dots & 0 \\ l_{31} & l_{32} & l_{33} & \dots & 0 \\ \dots & \dots & \dots & \dots & 0 \\ l_{n1} & l_{n2} & l_{n3} & \dots & l_{nn} \end{bmatrix}$$

Рисунок 6 – Вид матрицы L

Её элементы ищутся по формуле:

$$l_{ij} = a_{ij} - \sum_{k=1}^{j-1} l_{ik} * u_{kj}$$

U – Верхнеугольная матрица вида

$$U = \begin{bmatrix} 1 & u_{12} & u_{13} & \dots & u_{1n} \\ 0 & 1 & u_{23} & \dots & u_{2n} \\ 0 & 0 & 1 & \dots & u_{3n} \\ \dots & \dots & \dots & \dots & .. \\ 0 & 0 & 0 & \dots & 1 \end{bmatrix}$$

Рисунок 7 – Вид матрицы U

Её элементы ищутся по формуле:

$$u_{ij} = \frac{1}{l_{ii}}(a_{ij} - \sum_{k=1}^{i-1} l_{ik} * u_{kj})$$

Поиск должен быть выполнен в виде чередования столбцов. Сначала ищется весь 1 столбец L, затем 2й U и т.д.

После всех манипуляций мы преобразуем матрицы L и U в системы уравнений, которые необходимо решить:

$$L = \begin{cases} l_{11} * y_1 = B_1 \\ l_{21} * y_1 + l_{22} * y_2 = B_2 \\ l_{31} * y_1 + l_{32} * y_2 + l_{33} * y_3 = B_3 \\ l_{i1} * y_1 + l_{i2} * y_2 + l_{i3} * y_3 + l_{ij} * y_j = B_i \end{cases}$$

$$U = \begin{cases} u_{11} * x_1 + u_{12} * x_2 + u_{13} * x_3 + u_{ij} * x_j = y_1 \\ u_{22} * x_2 + u_{23} * x_3 + u_{24} * x_4 + u_{ij} * x_j = y_2 \\ u_{33} * x_3 + u_{34} * x_4 + u_{ij} * x_j = y_3 \\ u_{44} * x_4 + u_{ij} * x_j = y_i \end{cases}$$

*B – Элемент матрицы свободных членов.

После решения этих двух систем можно получить корни СЛАУ(x).

2.3 Описание ручного счета тестового примера

Даны матрицы:

$$A = \begin{matrix} 2 & 8 & 7 & 10 \\ 5 & 8 & 2 & 4 \\ 5 & 3 & 2 & 6 \\ 7 & 7 & 5 & 5 \end{matrix} \quad B = \begin{matrix} 17.8 \\ 10.5 \\ 10.1 \\ 12.7 \end{matrix}$$

Построим матрицы L и U по заданным формулам.

$$l_{11} = a_{11} - \sum_{k=1}^{1-1} l_{1k} * u_{k1} = 2 - 0 = 2$$

$$l_{21} = a_{21} - \sum_{k=1}^{1-1} l_{1k} * u_{k1} = 5 - 0 = 5$$

$$l_{31} = a_{31} - \sum_{k=1}^{1-1} l_{1k} * u_{k1} = 5 - 0 = 5$$

$$l_{41} = a_{41} - \sum_{k=1}^{1-1} l_{1k} * u_{k1} = 7 - 0 = 7$$

И дальше аналогично по формуле расчёта для L…

$$u_{12} = \frac{1}{l_{11}}\left(a_{12} - \sum_{k=1}^{1-1} l_{1k} * u_{k2}\right) = \frac{1}{2}(8 - 0) = 4$$

И дальше аналогично по формуле расчёта для U…

$$L = \begin{matrix} 2 & 0 & 0 & 0 \\ 5 & -12 & 0 & 0 \\ 5 & -17 & 6.464 & 0 \\ 7 & -21 & 7.632 & -7.96 \end{matrix} \quad U = \begin{matrix} 1 & 4 & 3.5 & 5 \\ 0 & 1 & 1.292 & 1.75 \\ 0 & 0 & 1 & 1.665 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Теперь построим систему уравнений по L матрице

$$\begin{cases} 2 * y_1 = 17.8 \\ 5 * y_1 - 12 * y_2 = 10.5 \\ 5 * y_1 - 17 * y_2 + 6.464 * y_3 = 10.1 \\ 7 * y_1 - 21 * y_2 + 7.632 * y_3 - 7.96 * y_4 = 12.7 \end{cases}$$

Решив систему уравнений, получаем:

$$y = \begin{cases} y_1 = 8.9 \\ y_2 = 2.83 \\ y_3 = 2.121 \\ y_4 = 0.8 \end{cases}$$

Строим систему уравнений по матрице U

$$\begin{cases} x_1 + 4 * x_2 + 3.5 * x_3 + 5 * x_4 = 8.9 \\ x_2 + 1.292 * x_3 + 1.75 * x_4 = 2.83 \\ x_3 + 1.665 * x_4 = 2.121 \\ x_4 = 0.8 \end{cases}$$

Решив систему уравнений, получаем:

$$x = \begin{cases} x_1 = 0.5 \\ x_2 = 0.4 \\ x_3 = 0.8 \\ x_4 = 0.8 \end{cases}$$

2.4 Разработка алгоритмов решения задачи

Функция находит корни СЛАУ методом LU разложения.

Входные данные:

- A – Матрица коэффициентов
- B – Матрица свободных членов

Выходные данные:

- x_s – Матрица корней

На рисунке 8 представлена блок-схема алгоритма решения СЛАУ LU разложением.

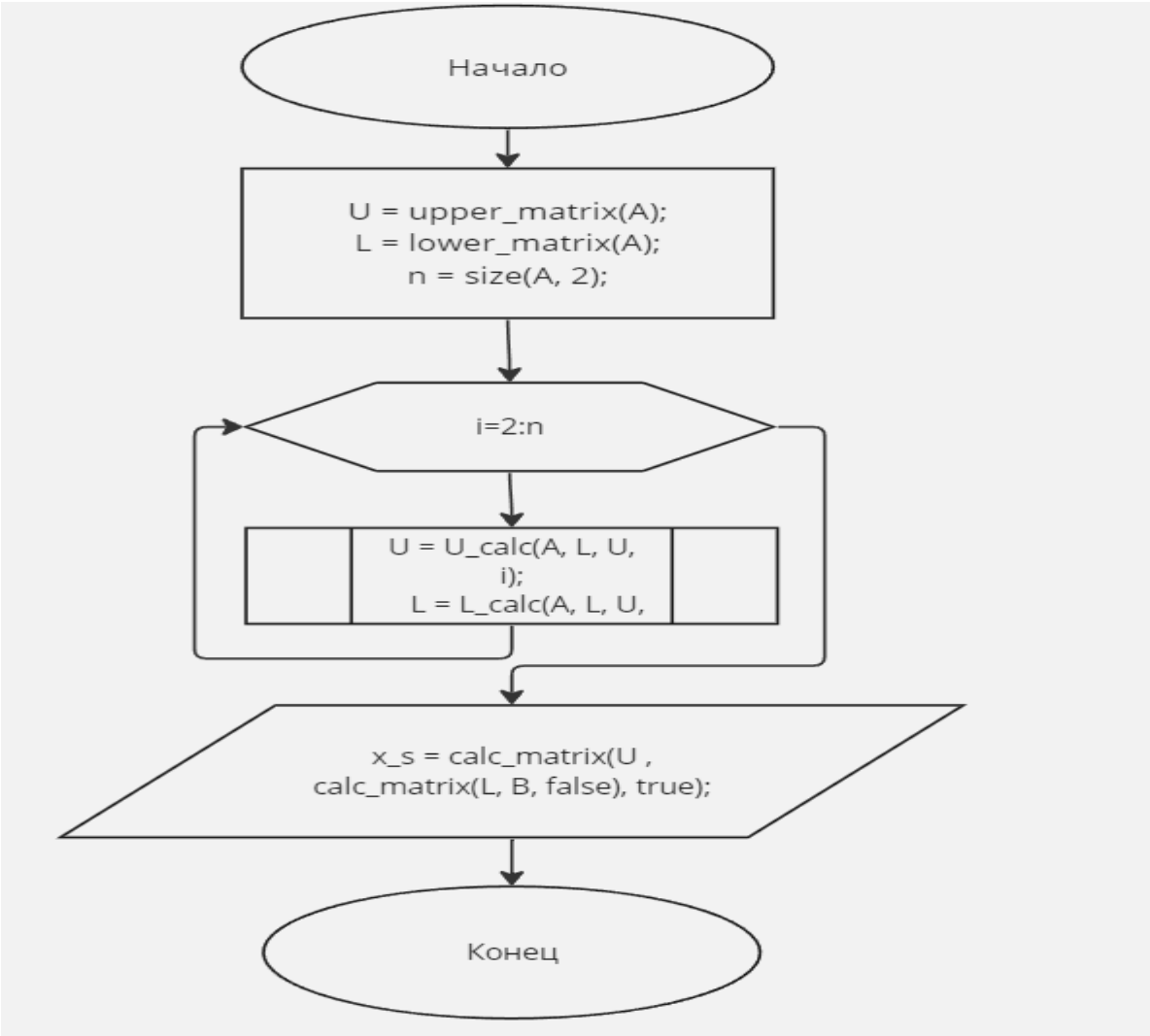


Рисунок 8 – блок схема SLAU_Solution

Функция вычленяет из передаваемой матрицы верхнеугольную матрицу.

Входные данные:

- A – Основная матрица

Выходные данные:

- U – Верхнеугольная матрица

На рисунке 9 представлена блок-схема алгоритма вычленения верхнеугольной матрицы из другой матрциы.

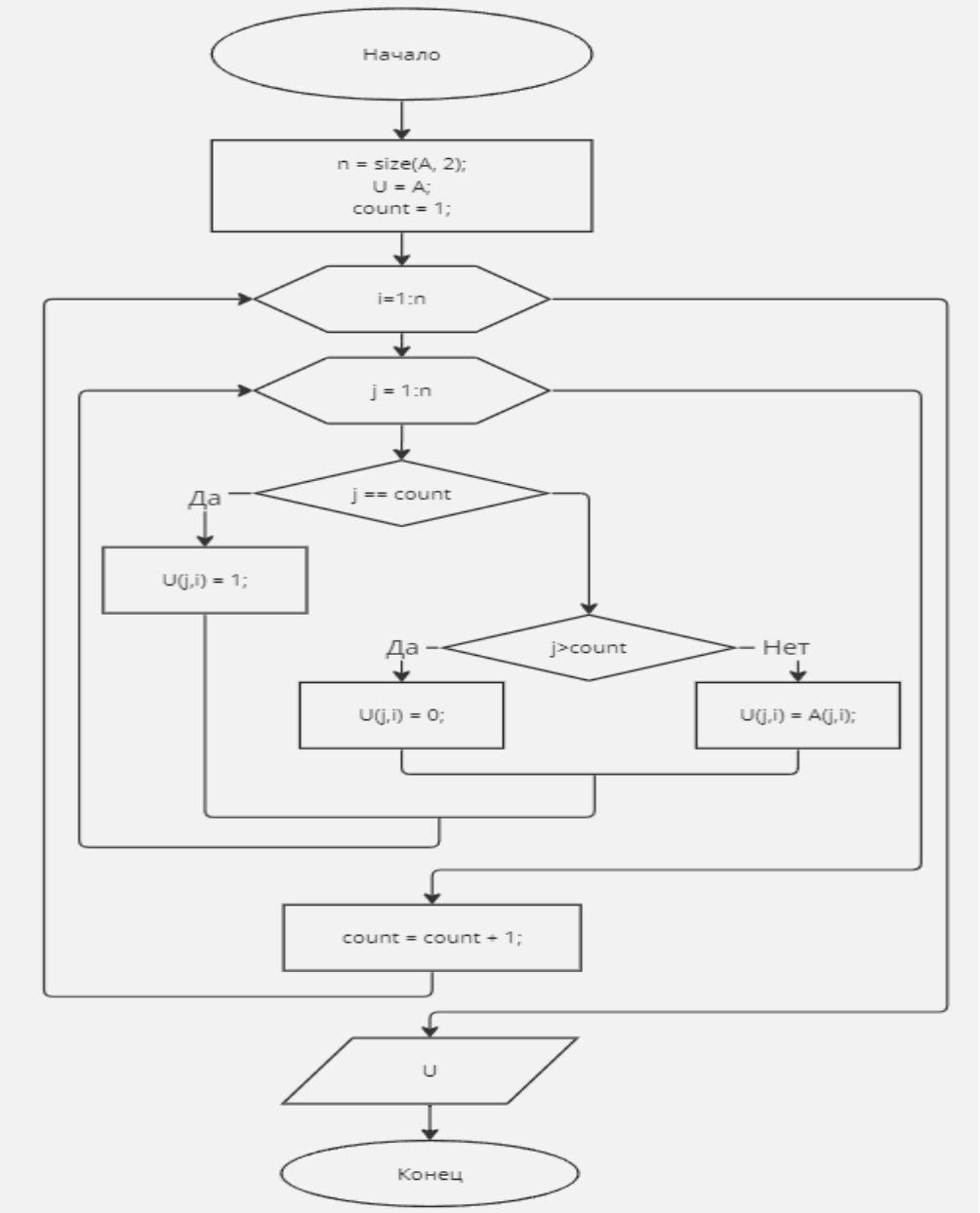


Рисунок 9 – Блок-схема upper_matrix

Функция вычленяет из передаваемой матрицы нижнеугольную матрицу.

Входные данные:

- A – Основная матрица

Выходные данные:

- U – Нижнеугольная матрица

На рисунке 10 представлена блок-схема алгоритма вычленения нижнеугольной матрицы из другой матрицы.

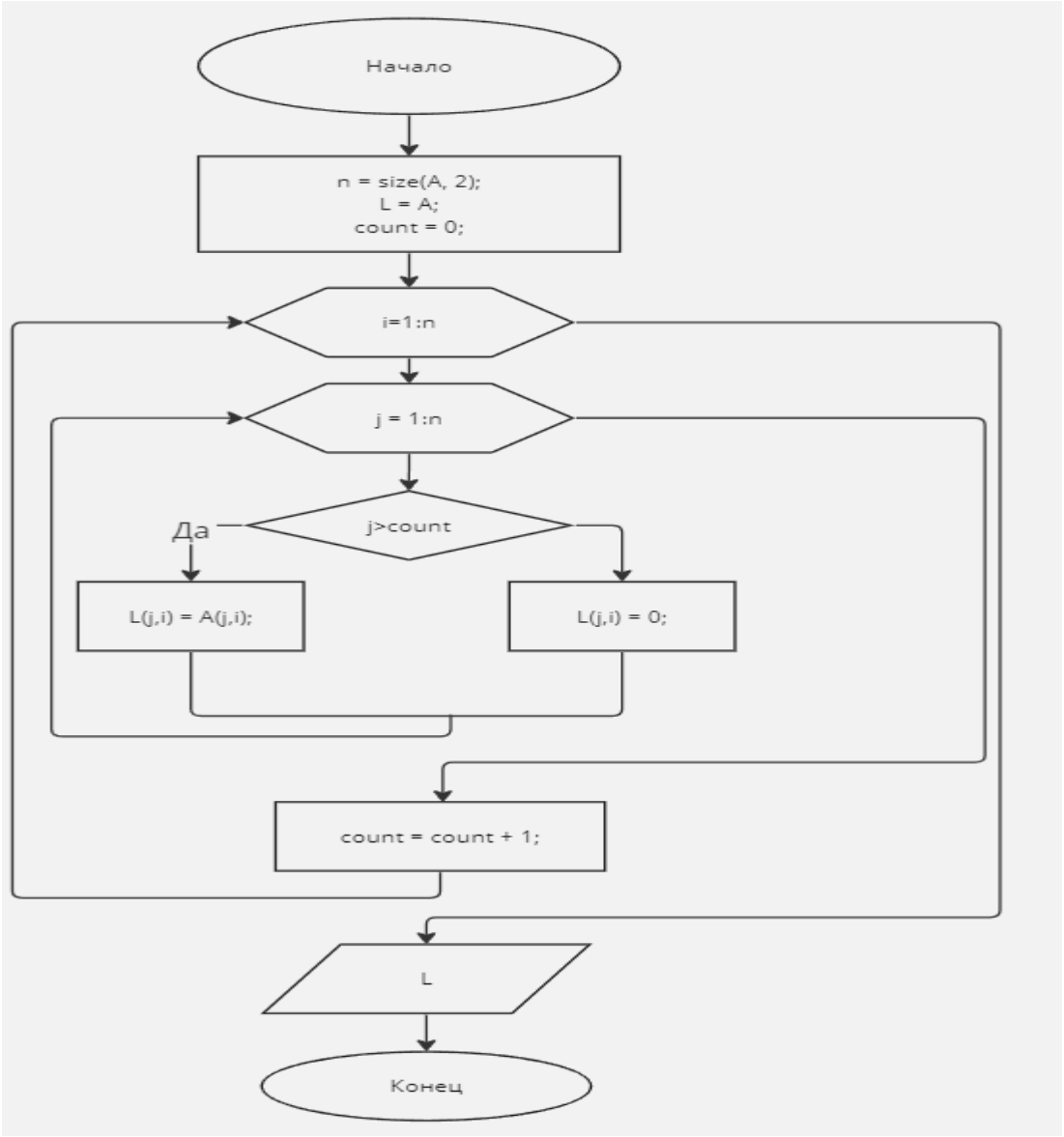


Рисунок 10 – Блок-схема lower_matrix

Функция высчитывает элементы U матрицы в n столбце.

Входные данные:

- A – Матрица коэффициентов
- L – L Матрица
- U – U Матрица
- n – Номер столбца

Выходные данные:

- U – U матрица

На рисунке 11 представлена блок-схема алгоритма высчитывания элементов U матрицы в n столбце.

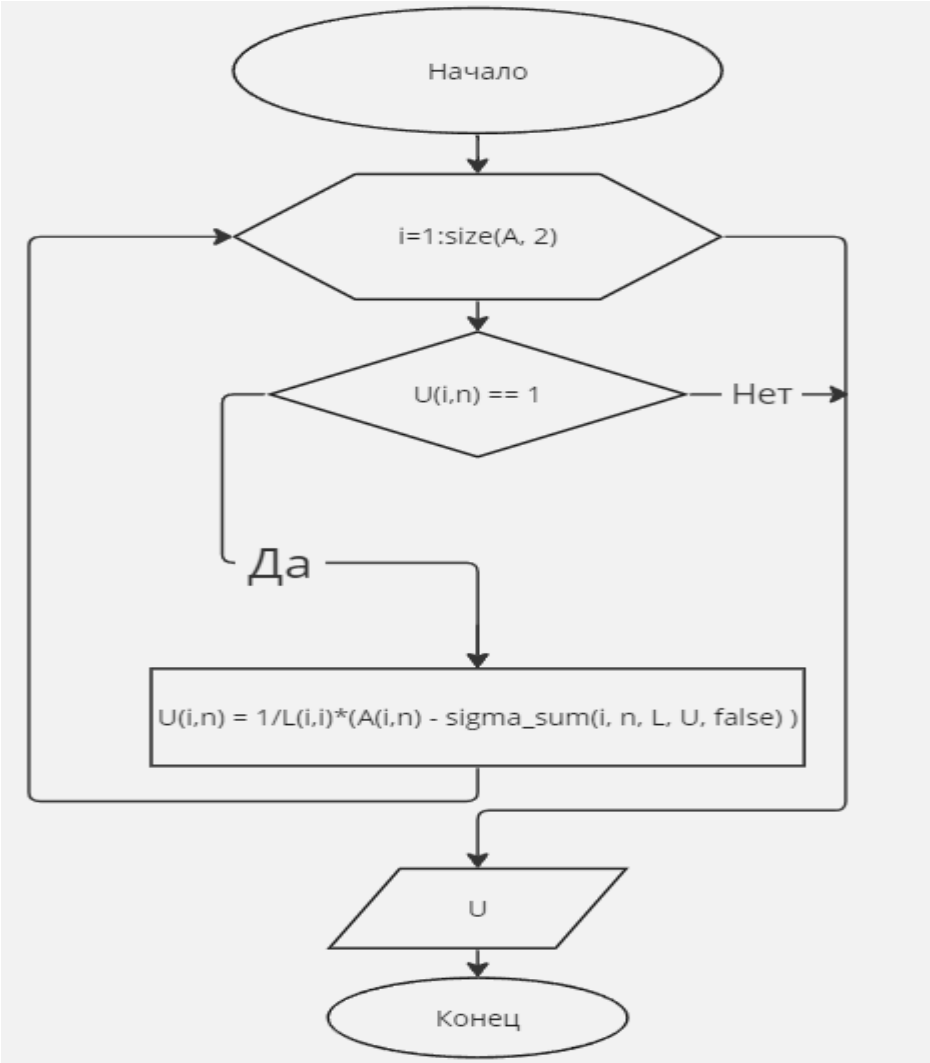


Рисунок 11 – Блок-схема U_calc

Функция высчитывает элементы U матрицы в n столбце.

Входные данные:

- A – Матрица коэффициентов
- L – L Матрица
- U – U Матрица
- n – Номер столбца

Выходные данные:

- L – L матрица

На рисунке 12 представлена блок-схема алгоритма высчитывания элементов L матрицы в n столбце.

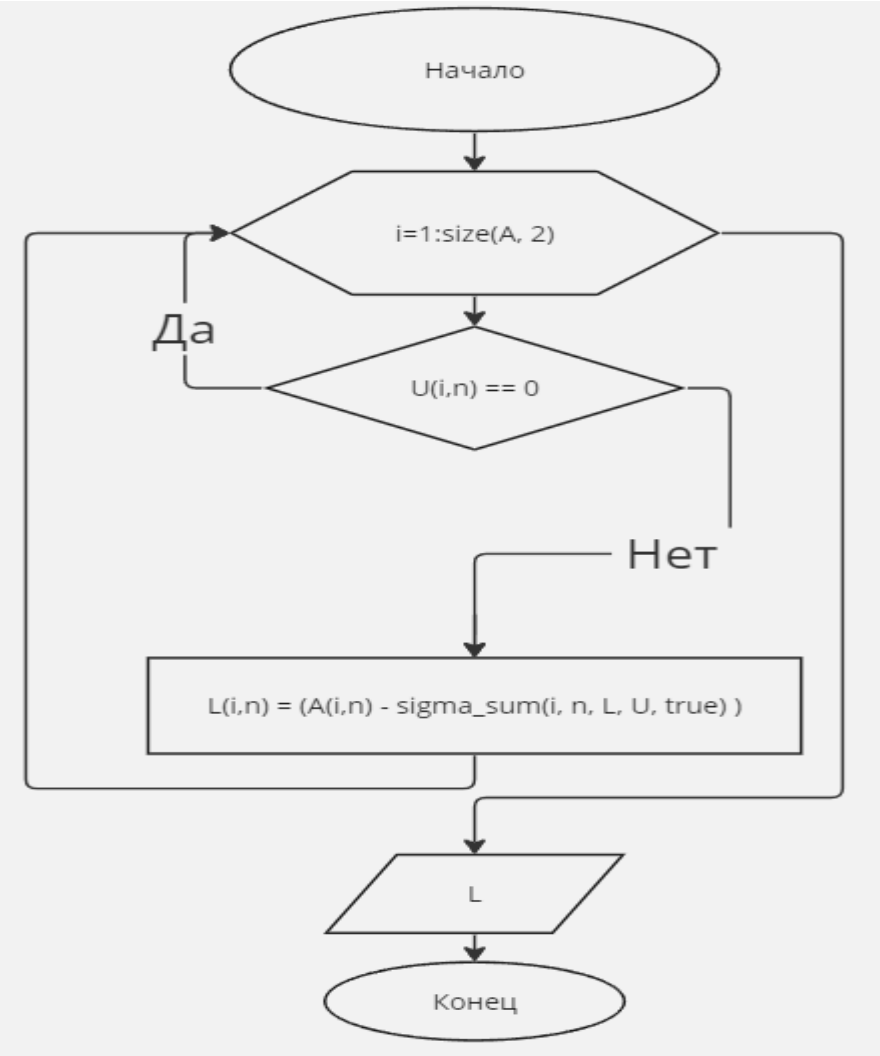


Рисунок 12 – Блок-схема L_calc

Функция считает значение по формуле lu разложения.

Входные данные:

- i,j - строка, столбец
- L,U - матрицы LU разложения
- bool - true(Формула для L) и false(Формула для U)

Выходные данные:

- sum – Значение элемента

На рисунке 13 представлена блок-схема алгоритма счёта значений по формула LU разложения.

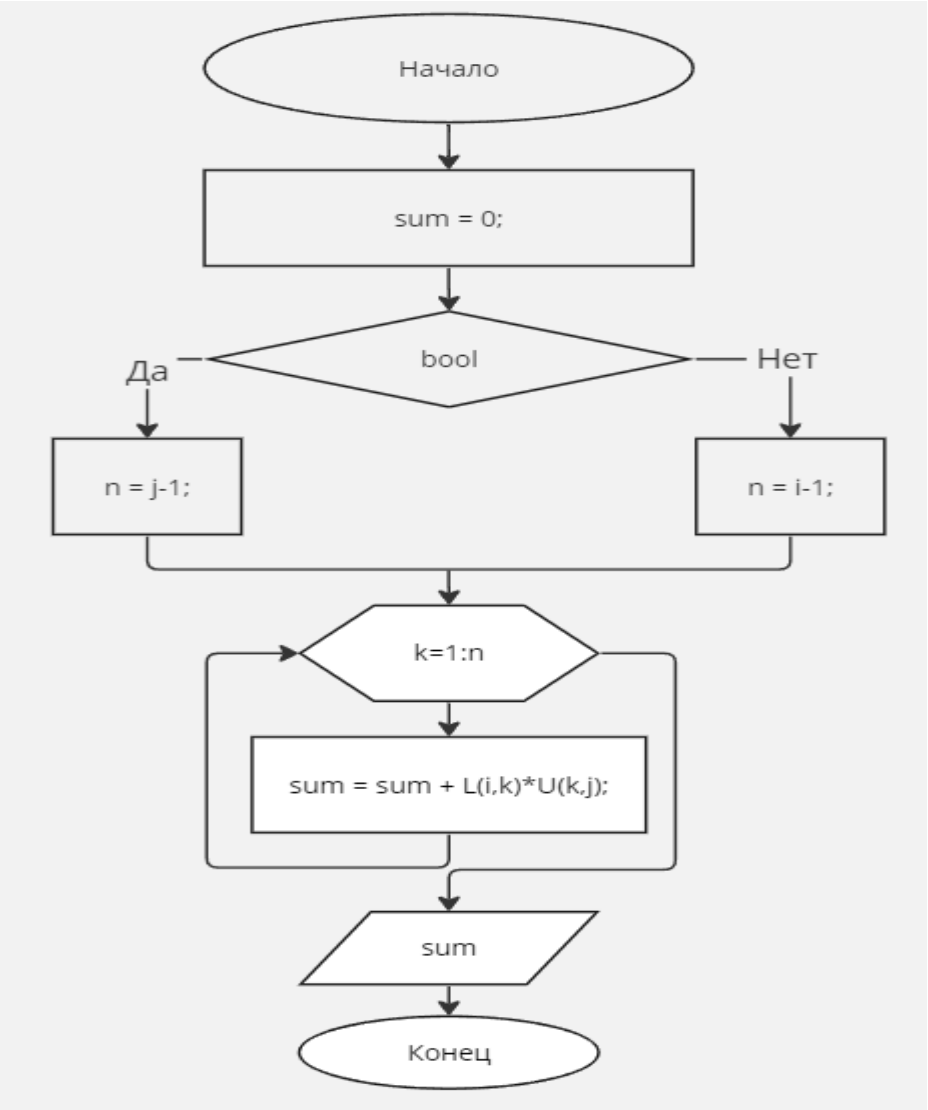


Рисунок 13 – Блок-схема sigma_sum

Функция систему уравнений вида L/U матрицы

Входные значения:

- A - матрица коэффициентов для системы
- B - матрица свободных членов для системы
- reversed - Система идёт сверху (L матрица)(false) или снизу (U матрица)(true)

Выходные данные:

- y – матрица корней для системы

На рисунке 14 представлена блок-схема алгоритма решения системы уравнения вила L или U матрицы.

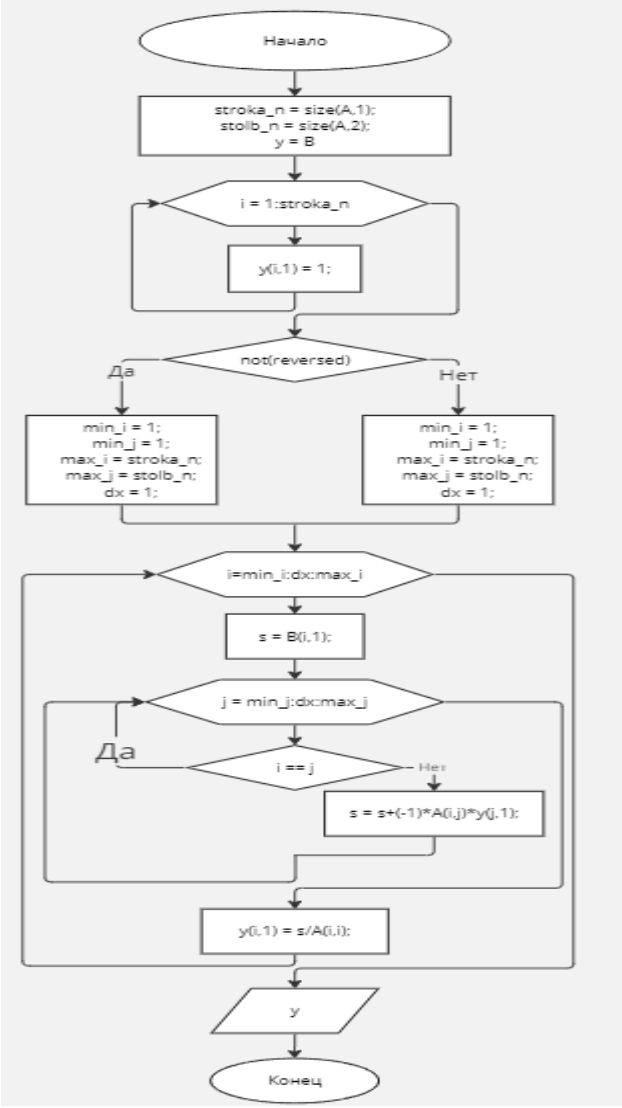


Рисунок 14 – блок-схема calc_matrix

2.4 Результат работы программы

На рисунке 15 изображена работа программы при выборе демонстрационного варианта.

```
Работа с данными пользователя.
Введите пункт для загрузки матрицы:
1. Загрузить файл .mat*
2. Ручной ввод
3. Отмена
*Примечание:
Должна быть матрицы A(коэффициенты) и B(свободные члены)
Способ 1
Введите название файла для загрузки: A
Заданная СЛАУ:
2*x1+8*x2+7*x3+10*x4=17.8
5*x1+8*x2+2*x3+4*x4=10.5
5*x1+3*x2+2*x3+6*x4=10.1
7*x1+7*x2+5*x3+3*x4=12.7
Корни СЛАУ по разработанной функции:
x1=0.5
x2=0.4
x3=0.8
x4=0.8
Корни СЛАУ по функции lu:
x1=0.5
x2=0.4
x3=0.8
x4=0.8
Время вычисления по разработанной функции:
   0.0000054691
Время вычисления по функции lu:
   0.0000021575
Нажмите любую кнопку для продолжения
```

Рисунок 15 – Пример работы программы на тестовой СЛАУ

На рисунке 16 изображена работа программы при вводе данных пользователем.

```
Работа с данными пользователя.
Введите пункт для загрузки матрицы:
1. Загрузить файл .mat*
2. Ручной ввод
3. Отмена
*Примечание:
Должна быть матрицы A(коэффициенты) и B(свободные члены)
Способ 1
Введите название файла для загрузки: AA
Заданная СЛАУ:
3*x1+4*x2+12*x3=9
3*x1+x2+6*x3=0
x1+2*x2+7*x3=10
Корни СЛАУ по разработанной функции:
x1=-5
x2=-3
x3=3
Корни СЛАУ по функции lu:
x1=-5
x2=-3
x3=3
Время вычисления по разработанной функции:
   0.0000040298
Время вычисления по функции lu:
   0.0000017970
Нажмите любую кнопку для продолжения
```

Рисунок 16 – Пример работы программы на СЛАУ пользователя

ТЕСТИРОВАНИЕ РАЗРАБОТАННОЙ ПРОГРАММЫ

В процессе учебной практики была разработана программа с текстовым интерфейсом. При запуске программы пользователя встречает главное меню, изображённое на рисунке 17.

```
Главное меню.
Введите номер пункта:
1. Решение нелинейных и трансцендентных уравнений
2. Решение СЛАУ
3. Информация о программе и авторе
4. Выход
Пункт
```

Рисунок 17 – Главное меню программы

Пользователю предлагается на выбор 4 пункта меню. 1 пункт меню (Решение уравнений) предлагает на выбор следующие варианты, изображённые на рисунке 18.

```
Решение нелинейных и трансцендентных уравнений.
Выберите дальнейшее действие.
Введите номер пункта:
1. Демонстрационный пример
2. Работа с данными пользователя
3. Назад
Пункт
```

Рисунок 18 – Решение уравнений

При выборе пункта 1, программа выводит пример на основе задания по варианту. Демонстрация изображена на рисунке 19.

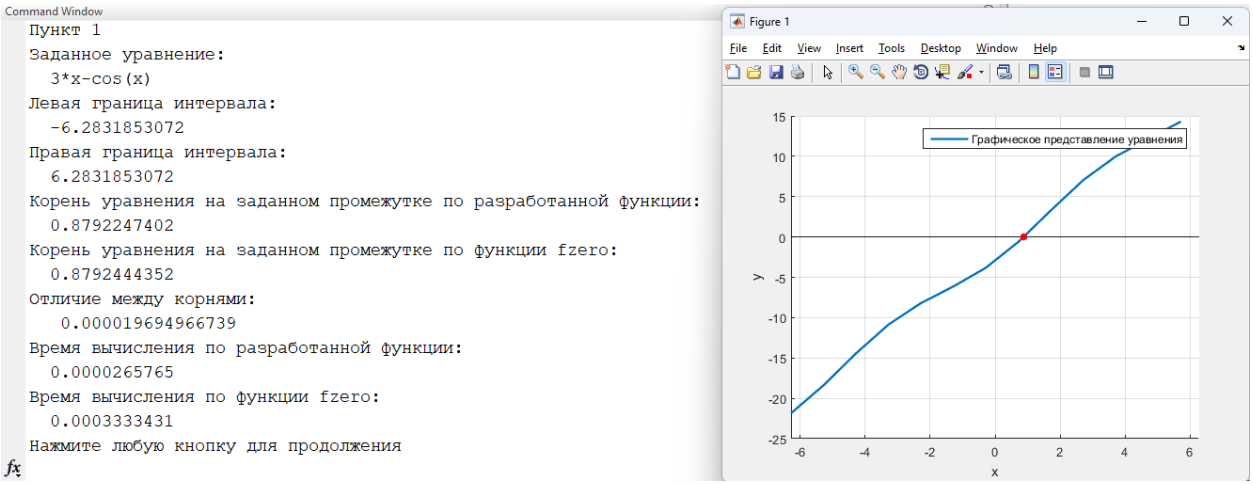


Рисунок 19 – Демонстрационный пример

При выборе пункта 2 пользователю предоставляется возможность вести функцию и промежуток для прорисовки графического отображения функции. Данные проверяются на корректность, функция должна существовать, а интервал должен быть корректно задан (Левая граница меньше правой, шаг меньше длины интервала). На рисунке 20-23 изображён некорректный ввод функции, числа, интервалов.

```
Работа с данными пользователя.
Введите данные для графика.
Введите функцию: f(x) = cor(x)
Введено некоректное значение.
Нажмите любую кнопку для продолжения
```

Рисунок 20 – Некорректная функция

```
Работа с данными пользователя.
Введите данные для графика.
Введите функцию: f(x) = cos(x)
Введите левую границу: 5t
Ошибка!
Данные содержат символ или пробел!
Введите левую границу:
```

Рисунок 21 – Некорректное число

```
Введите левую границу: 7
Введите правую границу: 8
Введите шаг: 10
Шаг больше промежутка
fx Введите левую границу:
```

Рисунок 22 – Некорректный шаг

```
Работа с данными пользователя.
Введите данные для графика.
Введите функцию: f(x) = cos(x)
Введите левую границу: 8
Введите правую границу: 4
Введите шаг: 10
Левая граница больше правой
fx Введите левую границу:
```

Рисунок 23 – Некорректные границы

На рисунке 24 изображён корректный ввод.

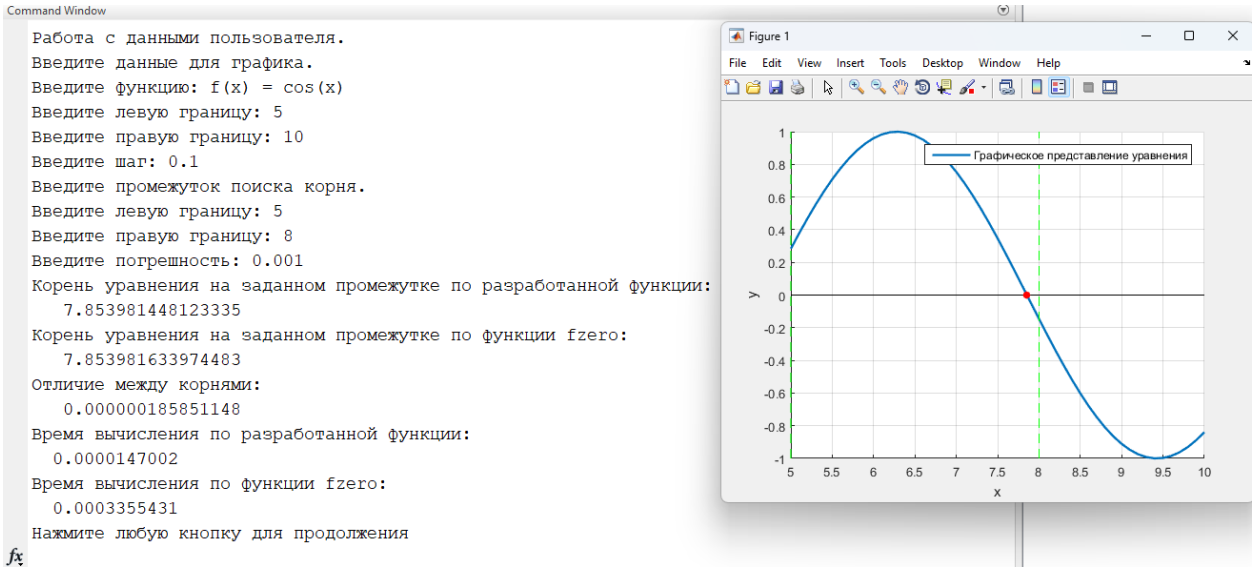


Рисунок 24 – Корректный ввод

При выборе пункта 2 в главном меню, которое изображено на рисунке 17 пользователю предлагается выбрать дальнейшие действия для решения СЛАУ. Это показано на рисунке 25.

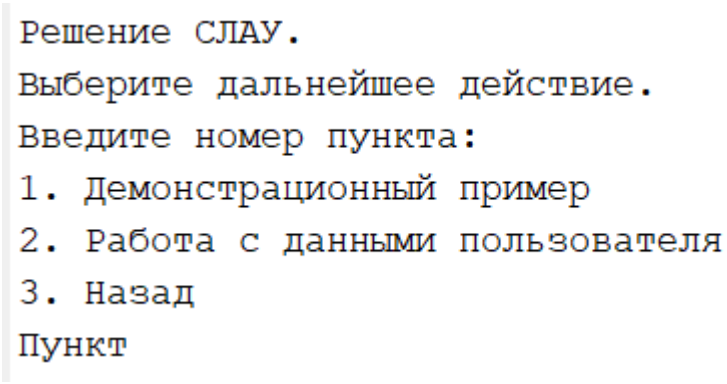


Рисунок 25 – Меню Решение СЛАУ

При выборе пункта 1 программа выводит демонстрационный пример на основе тестовых данных по варианту. Данный вывод показан на рисунке 26.

```
Решение СЛАУ.
Выберите дальнейшее действие.
Введите номер пункта:
1. Демонстрационный пример
2. Работа с данными пользователя
3. Назад
Пункт 1
Заданная СЛАУ:
2*x1+8*x2+7*x3+10*x4=17.8
5*x1+8*x2+2*x3+4*x4=10.5
5*x1+3*x2+2*x3+6*x4=10.1
7*x1+7*x2+5*x3+3*x4=12.7
Корни СЛАУ по разработанной функции:
x1=0.5
x2=0.4
x3=0.8
x4=0.8
Корни СЛАУ по функции lu:
x1=0.5
x2=0.4
x3=0.8
x4=0.8
Время вычисления по разработанной функции:
  0.0000122034
Время вычисления по функции lu:
  0.0000041657
Нажмите любую кнопку для продолжения
```

Рисунок 26 – Демонстрационный пример

При выборе пункта 2 в меню Решение СЛАУ (Рисунок 25) пользователю предлагается на выбор два способа загрузки данных. Это показано на рисунке 27.

```
Работа с данными пользователя.
Введите пункт для загрузки матрицы:
1. Загрузить файл .mat*
2. Ручной ввод
3. Отмена
*Примечание:
Должна быть матрицы A(коэффициенты) и B(свободные члены)
Способ
```

Рисунок 27 – Работа с данными пользователя.

При выборе пункта 1 пользователь должен ввести название файла, в котором содержатся матрицы A и B. Если файла не существует, то программа запрашивает название файла снова. Некорректный ввод показан на рисунке 28. Корректный ввод показан на рисунке 29.

```
Работа с данными пользователя.
Введите пункт для загрузки матрицы:
1. Загрузить файл .mat*
2. Ручной ввод
3. Отмена
*Примечание:
Должна быть матрицы A(коэффициенты) и B(свободные члены)
Способ 1
Введите название файла для загрузки: frg
Файла не существует
Для перехода обратно введите not
Введите название файла для загрузки:
```

Рисунок 28 – Вводимый файл не существует.

```
Введите название файла для загрузки: A
Заданная СЛАУ:
2*x1+8*x2+7*x3+10*x4=17.8
5*x1+8*x2+2*x3+4*x4=10.5
5*x1+3*x2+2*x3+6*x4=10.1
7*x1+7*x2+5*x3+3*x4=12.7
Корни СЛАУ по разработанной функции:
x1=0.5
x2=0.4
x3=0.8
x4=0.8
Корни СЛАУ по функции lu:
x1=0.5
x2=0.4
x3=0.8
x4=0.8
Время вычисления по разработанной функции:
  0.0000194554
Время вычисления по функции lu:
  0.0000113109
Нажмите любую кнопку для продолжения
fx
```

Рисунок 29 – Корректный ввод файла

При выборе пункта 2 на рисунке 27 пользователю предлагается на ввод количество уравнений в системе и коэффициент каждого неизвестного и свободный член. Числа проверяются на корректность, как и ранее. Пример корректного ввода показан на рисунке 30.

```
Способ 2
Введите количество уравнений в системе: 3
Уравнение 1
Введите коэффициент 1: 6
Введите коэффициент 2: 7
Введите коэффициент 3: 8
Введите свободный член 1: 3
Уравнение 2
Введите коэффициент 1: 4
Введите коэффициент 2: 3
Введите коэффициент 3: 6
Введите свободный член 2: 6
Уравнение 3
Введите коэффициент 1: 1
Введите коэффициент 2: 9
Введите коэффициент 3: 7
Введите свободный член 3: 5
Заданная СЛАУ:
6*x1+7*x2+8*x3=3
4*x1+3*x2+6*x3=6
x1+9*x2+7*x3=5
Корни СЛАУ по разработанной функции:
x1=-1.4659
x2=-1.3409
x3=2.6477
Корни СЛАУ по функции lu:
x1=-1.4659
x2=-1.3409
x3=2.6477
Время вычисления по разработанной функции:
```

Рисунок 30 – Корректный ввод с клавиатуры

При выборе пункта 3 на рисунке 17 программа показывают информацию о программе и авторе. Информация показана на рисунке 31.

```
Информация о программе и авторе.
Решение нелинейных и трансцендентных уравнений и СЛАУ.
Решение уравнений производится с помощью метода хорд.
Решение СЛАУ производится с помощью метода LU разложения.
Версия v1.0
Дата последнего обновления: 26.04.2024
Автор: Студент I курса ГУАП Ворошилов Д.Н.
Нажмите любую кнопку для продолжения
```

Рисунок 31 – Информация о программе и авторе

ЗАКЛЮЧЕНИЕ

Разработанная программа с текстовым интерфейсом реализует алгоритмы решения нелинейных уравнений методом хорд и СЛАУ методом LU разложения. Программа выполнена на языке программирования MATLAB. В программе предусмотрены всевозможные некорректные вводы пользователя. Приложение работает корректно и полностью выполняет свою цель. Точность результата решения нелинейных уравнений напрямую зависит от пользователя, вводящего данные для вычисления. Результат решения СЛАУ наиболее точен к истинным. В случае решения нелинейных уравнений разработанная функция работает быстрее, чем встроенная функция MATLAB. В СЛАУ быстрее выполняется встроенная функция lu() с последующим обратным делением, чем алгоритм решения СЛАУ методом LU разложения с помощью разработанных алгоритмов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Учебная ознакомительная практика. Применение численных методов при решении прикладных задач: учебно-методическое пособие / А. А, Фоменкова, М. А. Мурашова. – СПБ: ГУАП 2024 – 46 с.

2. Информатика. Алгоритмизация и структурное программирование в среде MATLAB: учеб. пособие / А. А. Ключарёв, А. А. Фоменкова, А. В. Туманова; под общ. ред. А. А. Ключарёва. – СПб.: ГУАП, 2019. – 143 с.

3. Типы и структуры данных в информатике и программировании: учеб. пособие / А. А. Ключарёв, А. А. Фоменкова. – СПб.: ГУАП, 2021. – 103 с.

ПРИЛОЖЕНИЕ 1

Листинг программного кода

begin.m

```
while true
clc
clear
close
disp('Главное меню.');
disp('Введите номер пункта:');
disp('1. Решение нелинейных и трансцендентных уравнений');
disp('2. Решение СЛАУ ');
disp('3. Информация о программе и авторе ');
disp('4. Выход ');

point = input('Пункт ', 's');

if point == '1'
    equation_lobby;
elseif point == '2'
    SLAU_lobby;
elseif point == '3'
    author_lobby;
elseif point == '4'
    disp('Работа программы завершена.');
    break
else
    begin;
    break;
end

end
```

autrhor_lobby.m

```
function author_lobby
    clc
    close
    disp('Информация о программе и авторе.');
    disp('Решение нелинейных и трансцендентных уравнений и СЛАУ.')
    disp('Решение уравнений производится с помощью метода хорд.')
    disp('Решение СЛАУ производится с помощью метода LU разложения.')
    disp('Версия v1.0')
    disp('Дата последнего обновления: 26.04.2024')
    disp('Автор: Студент I курса ГУАП Ворошилов Д.Н.')
    disp('Нажмите любую кнопку для продолжения ')
    input('');
    return;
end
```

equation_lobby.m

```
function equation_lobby
```

```
clc
clear
close
disp('Решение нелинейных и трансцендентных уравнений.');
disp('Выберите дальнейшее действие.');
disp('Введите номер пункта:');
disp('1. Демонстрационный пример');
disp('2. Работа с данными пользователя ');
disp('3. Назад ');

point = input('Пункт ', 's');

if point == '1'
    equation_demo;
elseif point == '2'
    equation_start;
elseif point == '3'
    begin;
    return;
else
    equation_lobby;
    return;
end

end
```

equation_start.m

```
function equation_start
clc
clear
close
disp('Работа с данными пользователя.')
disp('Введите данные для графика.')
fBody = input('Введите функцию: f(x) = ', 's');
try
    func = eval(['@(x)' fBody]);
    func(0);
catch
    disp('Введено некоректное значение.')
    SLAU_wait;
    equation_start;
end
[Xm, Xmx, dx] = interval_request(true);


x_s = Xm:dx:Xmx;
y_s = func(x_s);
hold on
grid on
plot(x_s,y_s,'LineWidth', 1.75);
legend('Графическое представление уравнения')
xlim([Xm Xmx]);
```

```
plot([Xm Xmx],[0 0] , 'color', 'black');
xlabel x
ylabel y

disp('Введите промежуток поиска корня.')
[Xmin, Xmax] = interval_request(false);

plot([Xmin Xmin],[min(y_s) max(y_s)], 'color', 'green', 'LineStyle', '--');
plot([Xmax Xmax],[min(y_s) max(y_s)], 'color', 'green', 'LineStyle', '--');

Epsilon = input('Введите погрешность: ');

y = @() equation_solution(Xmin, Xmax, func, Epsilon);
y_otvet = y();

if y_otvet == 'NO_SOL'
  disp('Корней нет/Невозможно решить методом хорд.');
  disp('Нажмите любую кнопку для продолжения ')
  input('');
  equation_lobby;
  return;
end

plot(y(),0,'r.','MarkerSize', 20)

y1 = @() fzero(func, [Xmin Xmax]);
y1_otvet = y1();

disp('Корень уравнения на заданном промежутке по разработанной функции: ');
fprintf('   %15.15f \n',  y_otvet);
disp('Корень уравнения на заданном промежутке по функции fzero: ');
fprintf('   %15.15f \n',  y1_otvet);
fprintf('Отличие между корнями: \n   %15.15f \n',  abs(y1_otvet - y_otvet));
disp('Время вычисления по разработанной функции: ');
fprintf('  %-15.10f \n',  timeit(y));
disp('Время вычисления по функции fzero: ');
fprintf('  %-15.10f \n',  timeit(y1));

disp('Нажмите любую кнопку для продолжения ')
input('');
equation_lobby;
end

equation_demo.m
function equation_demo
Xmin = -2*pi;
Xmax = 2*pi;
dx = 1;
Epsilon = 1.0e-04;
fBody = '3*x-cos(x)-2';
func = eval(['@(x)' fBody]);
x_s = Xmin:dx:Xmax;
```

```
y_s = func(x_s);

f1 = @() equation_solution(Xmin, Xmax, func, Epsilon);
f2 = @() fzero(func, [Xmin Xmax]);
f1_otvet = f1();

hold on
grid on
plot(x_s,y_s,'LineWidth', 1.75);
legend('Графическое представление уравнения')
xlim([Xmin Xmax])
plot([Xmin Xmax],[0 0] , 'color', 'black');
xlabel x
ylabel y
plot(f1_otvet,0,'r.','MarkerSize', 20)

disp('Заданное уравнение: ');
fprintf('  %-15.10s \n',  strcat(fBody,'=0'));
disp('Левая граница интервала: ');
fprintf('  %-15.10f \n',  Xmin);
disp('Правая граница интервала: ');
fprintf('  %-15.10f \n',  Xmax);
disp('Корень уравнения на заданном промежутке по разработанной функции: ');
fprintf('  %-15.10f \n', f1_otvet);
disp('Корень уравнения на заданном промежутке по функции fzero: ');
fprintf('  %-15.10f \n', f2());
fprintf('Отличие между корнями: \n   %15.15f \n',  abs(f2() - f1()));
disp('Время вычисления по разработанной функции: ');
fprintf('  %-15.10f \n',  timeit(f1));
disp('Время вычисления по функции fzero: ');
fprintf('  %-15.10f \n',  timeit(f2));

disp('Нажмите любую кнопку для продолжения ')
input('');
equation_lobby;
end

equation_solution.m
function [ x_new ] = equation_solution(Xmin, Xmax, func, Epsilon)
   %Находит локальный корень данного уравнения методом хорд
   %       Xmin - Число. Левая граница промежутка
   %       Xmax - Число. Правая граница промежутка
   %       func - Функция. Само уравнение
   %       Epsilon - Число. Погрешность подсчёта корня
   %       x_new - Число. Посчитанный корень

   if (func(Xmin) * func(Xmax)) > 0
      x_new = 'NO_SOL';
      return;
   end

   x_old = Inf;
```

```
    x_new = 0;
    while abs(x_new-x_old) >= Epsilon
        x_old = x_new;
        x_new = Xmin - (((Xmax-Xmin)*func(Xmin))/(func(Xmax)-func(Xmin)));
        Xmin = x_new;
    end
end

SLAU_demo.m
function SLAU_demo
A = [2 8 7 10;5 8 2 4;5 3 2 6;7 7 5 3];
B = [17.8; 10.5; 10.1; 12.7];


f1 = @() SLAU_Solution(A, B);
x_s = f1();

f2_1 = @() lu(A);
[L,U] = f2_1();
f2_2 = @() U \ (L \ B);
x_s_b = f2_2();

disp('Заданная СЛАУ: ');
SLAU_show(A,B);

disp('Корни СЛАУ по разработанной функции: ');
X_show(x_s)
disp('Корни СЛАУ по функции lu: ');
X_show(x_s_b)

warning('off','all')
disp('Время вычисления по разработанной функции: ');
fprintf('  %-15.10f \n',  timeit(f1));
disp('Время вычисления по функции lu: ');
fprintf('  %-15.10f \n',  timeit(f2_1)+timeit(f2_2));
warning('on','all')

disp('Нажмите любую кнопку для продолжения ')
input('');
clear;
SLAU_lobby;

end

SLAU_wait.m
function SLAU_wait
disp('Нажмите любую кнопку для продолжения ')
input('');
end

SLAU_start.m
function SLAU_start
```

```
% Данные по варианту
%A = [2 8 7 10;5 8 2 4;5 3 2 6;7 7 5 3];
%B = [17.8; 10.5; 10.1; 12.7];
clc
clear
close
disp('Работа с данными пользователя.')
disp('Введите пункт для загрузки матрицы:');
disp('1. Загрузить файл .mat*');
disp('2. Ручной ввод');
disp('3. Отмена ');
disp('*Примечание:');
disp('Должна быть матрицы A(коэффициенты) и B(свободные члены)');
file = input('Способ ', 's');

if file == '2'
    [A, B] = request_double_matrix();
elseif file == '1'
    str = input('Введите название файла для загрузки: ','s');
    m = fopen(strcat(str,'.mat'));
    while m == -1
        disp('Файла не существует')
        disp('Для перехода обратно введите not');
        str = input('Введите название файла для загрузки: ','s');
        if str == 'not'
            SLAU_start;
            break;
        end
        m = fopen(strcat(str,'.mat'));
    end
    fclose(m);
    m = matfile(strcat(str,'.mat'));
    A = m.A;
    B = m.B;
elseif file == '3'
    SLAU_lobby;
    return;
else
    SLAU_start;
    return;
end



if size(A, 2) ~= size(B, 1)
    disp('Матрицы разного размера!');
    SLAU_wait;
    SLAU_start;
    return
elseif size(A, 1) ~= size(A, 2)
    disp('Матрица коэффициентов неквадратная!');
    SLAU_wait;
```

```
    SLAU_start;
    return
end

rank_A = rang(A);
rank_AB = rang([A B]);

if rank_A == rank_AB
  if rank_A < size(A,1)
      disp('СЛАУ имеет бесконечное количество решений!');
      SLAU_wait;
      SLAU_start;
      return
  end
else
    disp('СЛАУ несовместна!');
    SLAU_wait;
    SLAU_start;
    return
end

if (deter(A)) == 0 || isnan(deter(A))
    disp('СЛАУ не имеет решений!');
    SLAU_wait;
    SLAU_start;
    return
end

A = Fix_diag_zeros(A);

f1 = @() SLAU_Solution(A, B);
x = f1();

f2_1 = @() lu(A);
[L,U] = f2_1();
f2_2 = @() U \ (L \ B);
x_better = f2_2();

disp('Заданная СЛАУ: ');
SLAU_show(A,B);
disp('Корни СЛАУ по разработанной функции: ');
X_show(x)
disp('Корни СЛАУ по функции lu: ');
X_show(x_better)

warning('off','all')
disp('Время вычисления по разработанной функции: ');
fprintf('  %-15.10f \n',  timeit(f1));
disp('Время вычисления по функции lu: ');
fprintf('  %-15.10f \n',  timeit(f2_1)+timeit(f2_2));
warning('on','all')
```

```
disp('Нажмите любую кнопку для продолжения ')
input('');
SLAU_lobby;
end
```

SLAU_Solution.m

```
function [ x_s ] = SLAU_Solution( A, B )
%Нахоидт корни СЛАУ методом LU разложения
%       A - Коэффициенты
%       B - Свободные члены
%       x_s - Корни уравнения
U = upper_matrix(A);
L = lower_matrix(A);

n = size(A, 2);

for i=2:n
   U = U_calc(A, L, U, i);
   L = L_calc(A, L, U, i);
end

x_s = calc_matrix(U , calc_matrix(L, B, false), true);

end
```

SLAU_show.m

```
function SLAU_show(A, B)
%Функция показывает СЛАУ в привычном виде
%   A - Коэффициенты
%   B - Свободные члены
str = '';
for i=1:size(A,1)
   for j=1:size(A,2)
      if A(i,j) ~= 0
         str_elem_pres = num2str(A(i,j));
         if A(i,j) ~= 1
            str = strcat(str,str_elem_pres);
            str = strcat(str,'*x');
         else
            str = strcat(str,'x');
         end
         num = num2str(j);
         str = strcat(str,num);
         if (j~=size(A,2))
            str_elem_fur = num2str(A(i,j+1));
            if (str_elem_fur(1) ~= '-') && (str_elem_fur(1) ~= '0')
               str = strcat(str,'+');
            end
         end
      end
   end
   str = strcat(str,'=',num2str(B(i,1)));
```

```
    disp(str)
    str = '';
end

end
```

SLAU_lobby.m

```
function SLAU_lobby
clc
clear
close
disp('Решение СЛАУ.');
disp('Выберите дальнейшее действие.');
disp('Введите номер пункта:');
disp('1. Демонстрационный пример');
disp('2. Работа с данными пользователя ');
disp('3. Назад ');

point = input('Пункт ', 's');

if point == '1'
    SLAU_demo;
elseif point == '2'
    SLAU_start;
elseif point == '3'
    begin;
    return;
else
    SLAU_lobby;
    return;
end

end
```

rang.m

```
function [count] = rang( A )
%Функция считает ранг матрицы A
%   A - матрица
B = A;
n = size(B,1);

i = 2;

while (B(1,1) == 0)
    B = swap(B, 1,i);
    i = i + 1;
    if i >= n
        break
    end
end

for i=1:n-1
```

```
    B = rang_help(B, i);
end

count = rang_help2(B);

end
```

rang_help.m

```
function [ A ] = rang_help( A, num )
%Функция работает по столбцу n и обнуляет столбец по неким элементом для
%получения треугольной матрицы
%   A - матрица
%   num - столбец
n = size(A,1);

i = num;
while A(i,num) == 0
    A = swap(A,num,i);
    i = i + 1;
    if i >= n
        break
    end
end
A = swap(A,num,i);

for j = num+1:n
    mnoj = A(j,num)/A(num,num);
    for h = num:size(A,2);
        A(j,h) = A(j,h)+(-1)*mnoj*A(num,h);
    end
end

end
```

rang_help.m

```
function [ count ] = rang_help2( A )
%Функция считает кол-во ненулевых строк в A
%   A - матрица
n = size(A,1);
num = size(A,2);

count = 0;

for i = 1:n
    for j = i:num
        if ((A(i,j) ~= 0) || (A(i,j) ~= -0))
            count = count + 1;
            break
        end
    end
end
```

```
end

number_request.m
function [ answer ] = number_request( request )
   %Запрашивает целое число и проверяет на корректность
   %       request - Будет показана при запросе числа
   %       answer - Вывод введённого числа, если оно удовлетворяет
   %       условию.
   answer = input(request, 's');
   if strfind(answer,',')
      disp('Ошибка!')
      disp('Данные содержат запятую!')
      answer = number_request(request);
      return;
   end
   answer = str2double(answer);
   if isnan(answer)
      disp('Ошибка!')
      disp('Данные содержат символ или пробел!')
      answer = number_request(request);
      return;
   end
end

request_double_matrix.m
function [ A, B] = request_double_matrix()
%Запрашивает матрицы коэффициентов и свободных членов построчно
A = [];
B = [];
n = number_request('Введите количество уравнений в системе: ');
if (n < 2)
    disp('Неверный ввод')
    SLAU_wait();
    request_double_matrix();
    return;
end

for i = 1:n
  stroka = ['Уравнение ' num2str(i)];
  disp(stroka)
  for j = 1:n+1
     if j > n
        stroka = ['Введите свободный член ' num2str(i) ': '];
        num = number_request(stroka);
        B(i,1) = num;
     else
        stroka = ['Введите коэффициент ' num2str(j) ': '];
        num = number_request(stroka);
        A(i,j) = num;
     end

  end
```

```
end
end
```

L_calc.m

```
function [ L ] = L_calc(A, L, U, n )
%Функция выделяет n столбец для L матрицы из A по формуле lu разложения
%   A - Матрица Коэффициентов
%   L - Текущая L матрица
%   U - U матрица
%   n - столбец где производится расччёт
for i=1:size(A, 2);
   if L(i,n) == 0
      continue
   end
   L(i,n) = (A(i,n) - sigma_sum(i, n, L, U, true) );
end

end
```

U_calc.m

```
function [ U ] = U_calc(A, L, U, n )
%Функция выделяет n столбец для U матрицы из A по формуле lu разложения
%   A - Матрица Коэффициентов
%   L - L матрица
%   U - Текущая U матрица
%   n - столбец где производится расччёт
for i=1:size(A, 2);
   if U(i,n) == 1
      break
   end
   U(i,n) = 1/L(i,i)*(A(i,n) - sigma_sum(i, n, L, U, false) );
end

end
```

Fix_diag_zeros.m

```
function [ L ] = Fix_diag_zeros( L )
%Функция проверяет наличие нулей на диагоналях и в случае нахождения
%прибавляет строку к строке с нулём для его избавления
%L - L матрица для поиска нулей
%L - L матрица без нулей на диагоналях
n  = size(L,1);
for i=1:n-1
   if L(i,i) == 0
      elem = L(i,i);
      while elem == 0
         for j=1:n
            if L(j,i) ~= 0
               elem = j;
               break;
            end
         end
      end
```

```
        for j=1:n
            L(i,j) = L(i,j) + L(elem,j);
        end
    end
end


end
```

X_show.m

```
function X_show(x_s)
%Функция показывает корни в читаемый вид
%   x_s - Матрица-вектор корней
for i=1:size(x_s,1)
    str = 'x';
    str = strcat(str,num2str(i));
    str = strcat(str,'=');
    str = strcat(str,num2str(x_s(i,1)));
    disp(str)
end

end
```

sigma_sum.m

```
function [ sum ] = sigma_sum(i,j, L, U, bool)
%Функция считает значение по формуле lu разложения
%   i,j - строка,столбец
%   L,U - матрицы LU разложения
%   bool - true(Формула для L) и false(Формула для U)

sum = 0;
if bool
    n = j-1;
else
    n = i-1;
end

for k=1:n
    sum = sum + L(i,k)*U(k,j);
end

end
```

swap.m

```
function [B] = swap(A, m, n)
%Функция меняет строки матрицы местами
%   A - Матрица
%   m,n - Какие строки поменять
B = A;
B([m n],:) = B([n m],:);
end
```

upper_matrix.m

```
function [ U ] = upper_matrix( A )
%Функция выделяет верхнеугольную матрицу из A
%   A - Исходная матрица
n = size(A, 2);

U = A;

count = 1;
for i=1:n
  for j = 1:n
     if j == count
        U(j,i) = 1;
     elseif j>count
        U(j,i) = 0;
     else
        U(j,i) = A(j,i);
     end
  end
  count = count + 1;
end
```

lower_matrix.m

```
function [ L ] = lower_matrix( A )
%Функция выделяет нижнеугольную матрицу из A
%   A - Исходная матрица
n = size(A, 2);

L = A;

count = 0;
for i=1:n
  for j = 1:n
     if j>count
        L(j,i) = A(j,i);
     else
        L(j,i) = 0;
     end
  end
  count = count + 1;
end
```